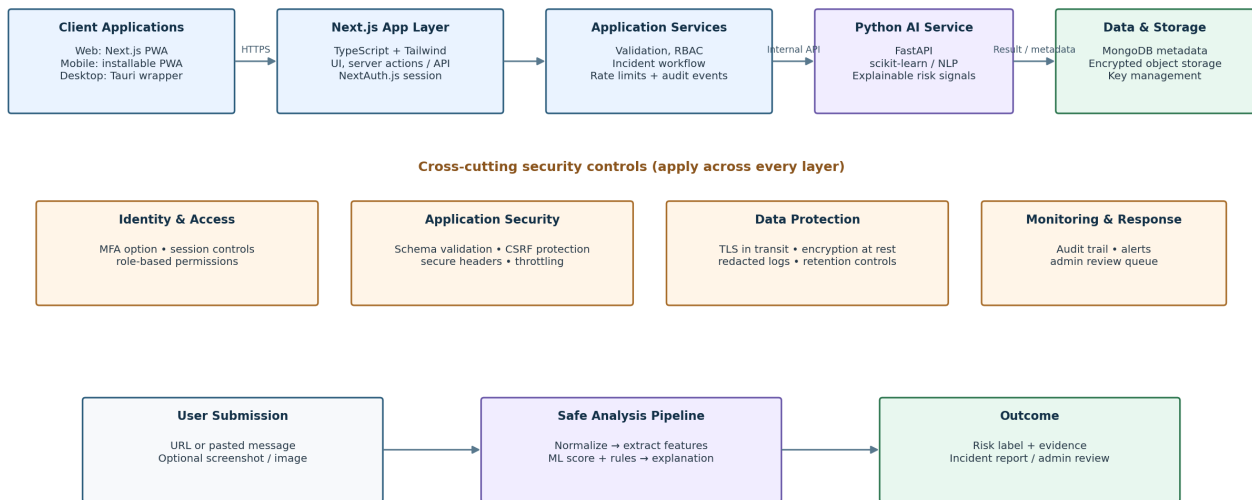


SecureSense AI

Cross-platform AI Cyber Threat Detection & Incident Management

SecureSense AI — System Architecture



Project category: Full-stack software engineering • Applied ML • Defensive cybersecurity • Cross-platform product design

Target delivery: Working prototype in approximately four months, with staged milestones and a measurable evaluation plan.

Primary goal: Help users assess suspicious URLs and messages, understand risk signals, and route incidents to authorized reviewers—without automatically opening suspicious links.

1. Problem Statement and Motivation

Problem statement. Individuals, students, and small organizations are frequently exposed to phishing URLs, impersonation messages, fraudulent login pages, and scam communications. Many users cannot reliably judge suspicious content, while small teams lack a single affordable workflow to analyze submissions, keep incident history, review recurring patterns, and coordinate response. Fragmented tools make it difficult to explain why an item was flagged, learn from false positives, or maintain an auditable record of decisions.

Proposed solution. SecureSense AI is a responsive web application with installable mobile access and a desktop wrapper. It accepts user-submitted URLs or message text, extracts safe features, combines a trained ML classifier with transparent security rules, and returns a risk category with supporting indicators and uncertainty. Users can report incidents and review their own history. Authorized analysts and administrators can triage reports, manage users and roles, tune approved thresholds, inspect audit events, and view aggregated trends.

Target users

Role	Primary needs
Individual / student	Check suspicious links or messages; understand risk signals; report an incident.
Small organization	Shared incident queue, repeat-pattern visibility, and consistent handling procedures.
Security analyst	Triage reports, record decisions, mark false positives, and escalate incidents.
Administrator / project owner	Manage accounts and roles, tune configuration, monitor usage, review audit logs.

Project objectives

- Build a full-stack application with a secure authentication and authorization model.
- Train and evaluate a baseline ML model on a documented, appropriately licensed dataset.
- Combine model output with deterministic rules and explainable evidence.
- Store reports and incidents with privacy-aware retention and auditability.
- Deliver web, mobile-installable, and desktop access from a shared codebase where practical.
- Measure model quality, latency, usability, and security behavior rather than claiming perfect detection.

Scope boundary: this is a defensive decision-support tool, not a guarantee that a URL is safe. It must show uncertainty, support user feedback, and avoid visiting suspicious URLs automatically.

2. Core Features and User Experience

User-facing features

1. URL analysis: normalize a URL, extract lexical/structural indicators, compare against approved local rules or an optional reputation feed, and display a risk category.
2. Message analysis: classify pasted text for phishing/scam indicators such as urgency, credential requests, impersonation language, and suspicious URL patterns.
3. Explainable result: display signals that contributed to the score, confidence/uncertainty, and safe next steps.
4. History and feedback: show previous submissions, allow users to report false positives, and track incident status.
5. Incident reporting: create a structured case with severity, category, timestamps, notes, and a status history.
6. Responsive cross-platform access: responsive Next.js UI, installable PWA for mobile/desktop browsers, and an optional Tauri desktop shell.

Admin / holder operations

- Manage accounts, deactivate compromised accounts, and assign roles.
- Review flagged reports and change case status with a recorded reason.
- Configure risk thresholds and approved rule lists with version history.
- View aggregate metrics: submissions, confirmed reports, false-positive feedback, response time, and model version.
- Review security audit events and authentication anomalies.
- Export a redacted incident report for project demonstration.
- Manage data-retention settings and revoke active sessions when required.

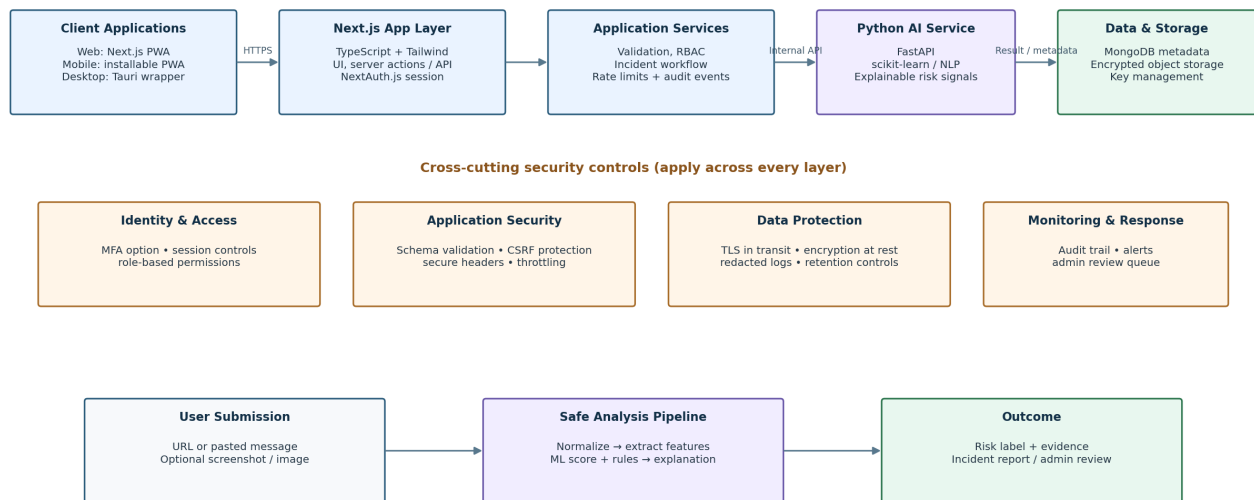
AI/ML modules (Python)

- URL classifier: begin with lexical features (URL length, suspicious tokens, number of subdomains, IP-address host, unusual character patterns) and a baseline model such as Logistic Regression or Random Forest. Add TF-IDF text classification for message content.
- Explainability: expose interpretable feature signals or model explanations; avoid presenting a score as certainty.
- Feedback loop: capture analyst/user labels for later offline evaluation, but do not silently retrain production models from unverified reports.
- Model evaluation: report precision, recall, F1, confusion matrix, and false-positive rate on a held-out dataset. Split data carefully to reduce leakage.

Recommended first version: support URL and pasted-message analysis first. Treat file scanning, browser extensions, real-time feeds, and automatic email integration as stretch goals, not launch blockers.

3. System Design

SecureSense AI — System Architecture



Architecture notes

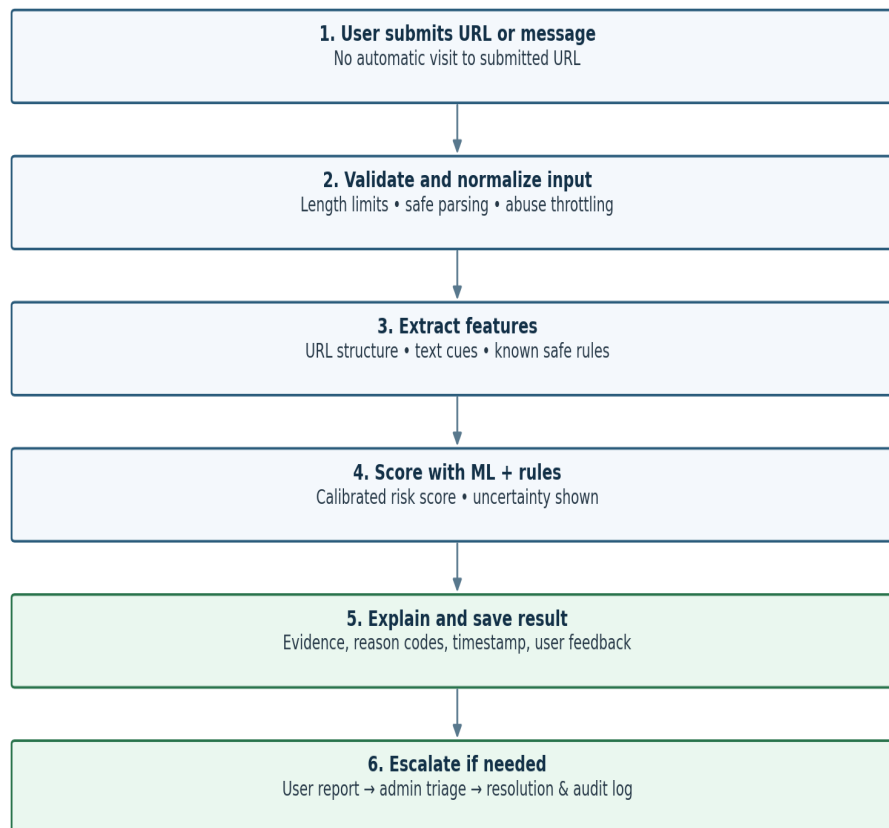
- Frontend: Next.js + TypeScript + Tailwind CSS. Use responsive layouts and accessible components.
- Authentication: NextAuth.js (Auth.js) with a well-supported provider or credentials flow. Hash passwords using Argon2id or bcrypt if local passwords are used; never store plaintext passwords.
- Application API: Next.js route handlers or a dedicated API layer handles validation, authorization, case workflows, and persistence.
- AI service: Python + FastAPI exposes a private internal endpoint for inference. Validate payloads and keep the service off the public network unless deliberately protected.
- Database: MongoDB is suitable for flexible incident records and analysis metadata. Use a separate object store only if the project later accepts attachments; do not store raw sensitive uploads unnecessarily.
- Security: HTTPS, server-side role checks, rate limits, secure cookies, CSRF protections where applicable, safe headers, audit logs, and secrets managed through environment variables.

Deployment choice

Use one responsive Next.js application as the primary product. Make it an installable Progressive Web App for mobile and desktop browser use. Package the same web experience in Tauri for a desktop executable if time permits. A separate native mobile application is an optional extension and should not be required to demonstrate the core product.

4. User Process and Incident Workflow

SecureSense AI — User & Incident Workflow



Risk decision flow

1. Validate input length and type; reject malformed or oversized submissions.
2. Normalize the URL/text and remove unsafe control characters where appropriate.
3. Extract local features without opening the submitted URL.
4. Run the ML model and deterministic rules; combine outputs using a documented decision policy.
5. Return category, contributing signals, model version, timestamp, and a safe-action recommendation.
6. If the user reports a threat, create an incident and route it to an analyst/admin queue.
7. Record review decisions and important administrative actions in an audit trail.

Important safety design: do not use a server-side HTTP request to fetch arbitrary submitted URLs. This can create server-side request forgery (SSRF) and expose internal services. If reputation lookups are added, use a reputable external API with strict controls, or a preloaded/offline dataset for the initial prototype.

5. Folder Structure

The structure below separates the user interface, application API, Python inference service, database configuration, shared types, tests, and documentation. It is intentionally modular but still manageable for a student project.

```
securesense-ai/
├── apps/
│   ├── web/                                # Next.js + TypeScript + Tailwind
│   │   ├── app/
│   │   │   ├── (auth)/                    # sign-in, sign-up, recovery
│   │   │   ├── (dashboard)/              # protected user pages
│   │   │   ├── admin/                    # admin-only screens
│   │   │   ├── api/                      # route handlers / webhooks
│   │   │   ├── layout.tsx
│   │   │   └── page.tsx
│   │   ├── components/                  # reusable UI
│   │   ├── lib/                         # auth, API client, validation
│   │   ├── middleware.ts                # coarse route protection only
│   │   ├── public/                     # PWA icons and manifest
│   │   └── tests/
│   └── desktop/                         # optional Tauri shell
├── services/
│   ├── ml-api/
│   │   ├── app/
│   │   │   ├── main.py                  # FastAPI entry point
│   │   │   ├── routes/                  # inference endpoints
│   │   │   ├── schemas/                 # Pydantic request/response models
│   │   │   ├── features/                # URL/text feature extraction
│   │   │   ├── models/                  # training and inference code
│   │   │   ├── explain/                 # reason codes / explanations
│   │   │   ├── security/                # validation, request limits
│   │   │   ├── model_artifacts/         # versioned model files (not secrets)
│   │   │   ├── training/
│   │   │   ├── tests/
│   │   └── requirements.txt
│   └── packages/
├── shared-types/                         # shared API contracts / schemas
├── database/
│   ├── indexes/                         # MongoDB index definitions
│   ├── seed/                           # synthetic demo records
│   └── migrations-notes/
├── docs/
│   ├── problem-statement.md
│   ├── system-design.md
│   ├── threat-model.md
│   ├── api-spec.md
│   └── evaluation-report.md
├── scripts/                             # safe setup/evaluation helpers
├── .env.example                         # placeholders only
├── .gitignore
├── docker-compose.yml                  # local development
├── README.md
└── LICENSE
```

Implementation note

Start with a single repository and a small set of services. Do not create dozens of packages before a working end-to-end flow exists. The desktop wrapper, live threat feeds, and optional native mobile app can be added after the web/PWA path works.

6. Data Model and API Design

Suggested collections (MongoDB)

Collection	Important fields / purpose
users	id, email, passwordHash/providerId, role, status, createdAt, lastLoginAt
analyses	id, userId, inputType, redactedInput, riskLabel, score, reasonCodes, modelVersion, createdAt
incidents	id, analysisId, reporterId, assignedTo, severity, status, resolution, updatedAt
incident_events	id, incidentId, actorId, action, reason, timestamp; append-only audit-style history
security_audit_logs	id, actorId, action, targetType, targetId, result, timestamp, redactedMetadata
model_versions	id, version, datasetVersion, metrics, deployedAt, enabled
system_settings	settingKey, settingValue, updatedBy, updatedAt, version

Example API endpoints

POST /api/analyses — submit URL/text for analysis.

GET /api/analyses/:id — retrieve an authorized analysis result.

POST /api/incidents — report a suspected threat.

GET /api/incidents — list cases visible to the current role.

PATCH /api/incidents/:id — analyst updates status with a recorded reason.

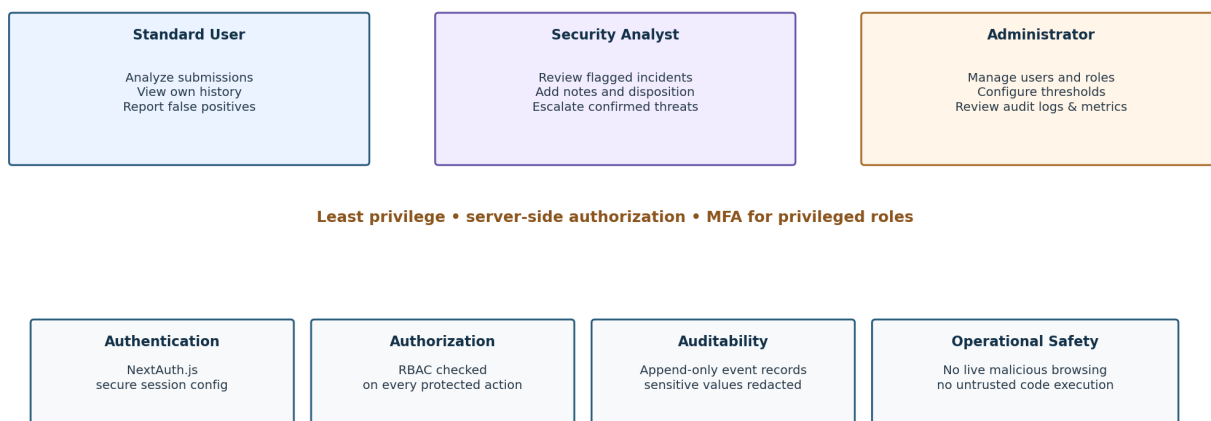
GET /api/admin/metrics — aggregate metrics for authorized admins.

POST /internal/inference — private Next.js-to-Python inference request.

Every protected endpoint must verify the current identity and role on the server. Hiding an admin button in the UI is not authorization. Enforce ownership checks so users cannot fetch another user's analysis by changing an ID.

7. Cybersecurity Controls and Threat Model

Roles, Admin Operations & Security Boundaries



Minimum security checklist

- **Authentication:** secure sessions, strong password hashing if applicable, session expiry and revocation, optional MFA for administrators.
- **Authorization:** server-side RBAC and object-level ownership checks on every protected operation.
- **Input handling:** schema validation, strict request-size limits, URL parsing with safe libraries, output encoding, and no unsafe shell execution.
- **Abuse resistance:** per-user/IP rate limits, login throttling, request quotas, and generic authentication error messages.
- **Browser protections:** HTTPS, secure HttpOnly cookies, appropriate SameSite setting, CSP and other security headers; CSRF controls as applicable to the chosen auth pattern.
- **Data safety:** minimize raw content storage, redact sensitive data from logs, define retention/deletion rules, and encrypt backups. Use managed keys where feasible; do not hardcode encryption keys.
- **Auditability:** log privileged actions and incident decisions; protect audit records against ordinary user modification.
- **Dependency hygiene:** lock dependency versions, run dependency scanning, keep secrets out of Git, and document how to rotate credentials.
- **AI-specific risks:** adversarial input, model uncertainty, data leakage in logs, poisoned feedback, and overconfident false positives.
- **Safe operations:** never auto-open user-submitted URLs from the backend; don't download or execute submitted files in the MVP.

Security testing plan

Test unauthorized access to admin routes, cross-user object access, malformed and oversized payloads, brute-force throttling, session revocation, injection attempts, secret leakage in logs, and dependency vulnerabilities. Document findings and fixes in a small threat-model report.

8. Four-Month Build Plan

Period	Build goals	Definition of done
Weeks 1–2	Requirements, wireframes, repo setup, database and authentication	Skeleton works; basic user and role model; README and architecture committed
Weeks 3–4	Analysis UI, API contracts, safe URL/text feature extraction	User can submit an item and receive a deterministic baseline result
Weeks 5–6	Dataset preparation and ML baseline	Reproducible training script, held-out metrics, model card, error analysis
Weeks 7–8	Python FastAPI inference service + explanation layer	Web app calls private inference API and shows reason codes
Weeks 9–10	Incident workflow and analyst/admin dashboard	Report, assign, resolve, and audit an incident with role checks
Weeks 11–12	Security hardening and evaluation	Rate limits, authorization tests, dependency scan, threat model
Weeks 13–14	PWA polish, responsive layouts, optional Tauri desktop wrapper	Deployable mobile experience and tested desktop build if time allows
Weeks 15–16	Deployment, documentation, demo dataset, presentation	Reproducible setup, demo video, evaluation report, final-year presentation

Prioritization rule

A strong working MVP is better than many unfinished features. The must-have path is: sign in → submit URL/message → get explainable analysis → report incident → authorized analyst reviews it → audit event is saved. Build and test this end-to-end before adding integrations.

9. Evaluation, Deliverables, and Risks

How to demonstrate project quality

- ML: precision, recall, F1, false-positive rate, confusion matrix, inference latency, and error analysis on a held-out set.
- Product: time to complete a check, task completion rate in a small usability test, and accessibility/responsive checks.
- Security: pass/fail results for role-based access, cross-user data isolation, rate limiting, session revocation, and dependency scanning.
- Reliability: API error rates, median/p95 latency under a small reproducible load test, and behavior when the model service is unavailable.
- Reproducibility: documented data source/licensing, environment setup, model training steps, sample inputs, and known limitations.

Expected final deliverables

1. Working Next.js application with user and admin roles.
2. Python FastAPI model-inference service and reproducible training pipeline.
3. Database schema/index notes and safe synthetic demo data.
4. Architecture, process-flow, and threat-model diagrams.
5. Automated tests for API behavior and authorization.
6. Evaluation report with real metrics and limitations.
7. README, setup instructions, demo video, and final presentation slides.

Main risks and mitigation

- Dataset bias or weak labels: document dataset limitations and evaluate on a held-out set.
- False positives: show reasons and uncertainty, collect feedback, and keep human review for important decisions.
- Cross-platform scope creep: deliver responsive PWA first; treat native mobile and desktop packaging as staged enhancements.
- Security feature overreach: implement a small number of controls correctly and test them, rather than claiming enterprise-grade protection.
- Sensitive data: use synthetic demo content, minimize retention, and avoid storing complete raw messages unless needed and clearly disclosed.

Suggested project tagline: "Detect the signal. Explain the risk. Manage the response."

Note: This blueprint is a proposed design. Detection performance, security effectiveness, and deployment readiness must be established by implementation and testing; they are not guaranteed by the architecture alone.